

Apuntes Clase 2 Semana 11

Jose Maria Vindas Ortiz
2022209471

*Escuela de Ingeniería en Computadores
Instituto Tecnológico de Costa Rica*

Profesor: Steven Pacheco Portuquez
Curso: IC6200 Inteligencia Artificial
Fecha: 7 de mayo de 2026

Resumen—En esta lección se inició el tema de Autoencoders, una arquitectura capaz de reconstruir la entrada a partir de un vector reducido, se procedió a ver que tareas puede resolver esta arquitectura, que partes componen el autoencoder, hiper parámetros a considerar al trabajar con ellos y finalmente que tipos de autoencoders existen.

I. ENTREGA DEL PROYECTO

Antes de empezar la lección, a petición de los estudiantes se solicitó el aplazamiento de la fecha de entrega del proyecto, el profesor al confirmar que la mayoría de grupos tienen una buena cantidad de avances, acepto aplazar la entrega al domingo 10 de mayo a las 7pm, puesto se necesita un margen de tiempo para revisar los informes, a cambio de las siguientes condiciones:

- Informe tiene que presentar SOTA.
- Muestren gráficas comparativas de Training y Validation de los modelos (overfitting) WandB
- Análisis matriz de confusión vs los resultados obtenidos con el mobile.

Estas condiciones, además de hablarse al principio de la lección, fueron pasadas por el grupo de WhatsApp de la clase.

II. INTRODUCCIÓN

Lo que se ha visto hasta el momento en clase son arquitecturas en las que se utiliza una red capaz de procesar una imagen mediante una CNN, como LeNet, ResNet y MobileNet, produciendo un vector reducido con información semántica condensada, el cual es enviado a un clasificador.

Ahora se procederá a estudiar una arquitectura que no clasifica, sino que reconstruye: el Autoencoder.

III. AUTOENCODER

El objetivo de un autoencoder es reconstruir la entrada, haciéndola pasar por una arquitectura convolucional que la reduzca a un vector y, a partir de este, volver a construir la misma entrada con ligeras variaciones.

Se debe tomar en cuenta que se continuará con el tema de procesamiento de imágenes, por lo que todos los siguientes ejemplos estarán relacionados con la reconstrucción de imágenes.

En teoría, un autoencoder utiliza "unsupervised learning"; sin embargo, en la práctica sí existe supervisión durante el aprendizaje, ya que se emplean los mismos mecanismos

utilizados en procesos de supervised learning. La diferencia radica en que la etiqueta objetivo corresponde a la misma entrada, la cual se compara con el resultado obtenido. En otras palabras, sí se está controlando cuál debería ser la salida mediante un punto de comparación.

Existen otros tipos de autoencoders, por ejemplo, aquellos que producen un μ y un σ para generar un espacio latente continuo orientado a la generación de imágenes. El objetivo es que todas las imágenes sigan una distribución, usualmente normal. Estas imágenes pueden extrapolarse, lo que permite la creación de datos sintéticos nunca antes vistos.

IV. TAREAS A RESOLVER

A. Reduccion de dimencionalidad

Permite reducir el espacio utilizado para almacenar imágenes en una base de datos. Imágenes que normalmente ocuparían una gran cantidad de espacio son convertidas en vectores más pequeños, los cuales requieren únicamente una fracción del espacio que utilizaría una imagen convencional y que, además, pueden reconstruirse nuevamente en sus imágenes originales en cualquier momento.

Además, permite la comparación de imágenes, ya que los espacios latentes de imágenes similares también serán muy cercanos entre sí. Incluso, se obtienen representaciones vectoriales de mayor calidad que las producidas por algoritmos como PCA, por ejemplo.

B. Deteccion de anomalias

Se puede entrenar un autoencoder para reconstruir únicamente datos correctos y, de esta manera, detectar datos anómalos o incorrectos.

Por ejemplo, si se introduce un voucher bancario en un autoencoder entrenado para procesar vouchers legítimos, este podrá reproducirlo sin problemas.

Sin embargo, al introducir un voucher fraudulento, se generará un vector latente de baja calidad y se producirá una imagen muy diferente a la de entrada, mostrando un error $(x - \hat{y}^2)$ alto en la reconstrucción, lo que indica una señal de fraude.

C. Procesamiento de imágenes

Consiste en restaurar imágenes deterioradas y producir imágenes digitales de mejor calidad.

Se toma un dataset y, antes de introducir las imágenes al autoencoder, se les aplica un filtro dependiendo del objetivo deseado: ruido, escala de grises, deterioro, entre otros. Posteriormente, se entrena el autoencoder para reconstruir la imagen original antes de aplicar dicho filtro, permitiendo obtener un modelo capaz de restaurar imágenes.

Incluso es posible aumentar la resolución de imágenes. Para ello, se aplica previamente un filtro que reduzca la resolución y se entrena el modelo para revertir dicho proceso; es decir, realizar "Super Resolution", generando imágenes de alta resolución a partir de imágenes de baja resolución.

V. PARTES DEL AUTOENCODER

- Encoder: Red convolucional encargada de reducir la imagen de entrada a un vector de menor dimensión.
- Latent Space (cuello de botella): Vector reducido que contiene información semántica condensada.
- Decoder: Red encargada de tomar el vector latente y reconstruir la imagen, utilizando operaciones inversas a la convolución.

A. Encoder

Conjunto de bloques convolucionales seguidos de módulos de pooling, encargados de extraer las características de la imagen de entrada y comprimir su información.

El objetivo es condensar la imagen en un vector y aprender la distribución de dicho vector, proceso conocido como downsampling.

La obtención de una representación latente es uno de los principios fundamentales detrás de gran parte de la inteligencia artificial moderna. La capacidad de distribuir distintos datos dentro de un espacio latente permite separar datos diferentes y agrupar datos similares.

Esto brinda la posibilidad de navegar dicho espacio y encontrar representaciones nunca antes vistas. De hecho, gran parte de los principios de la IA generativa se basan en este tipo de espacios latentes.

B. Cuello de botella

Vector resultante del proceso convolucional. Es la parte más crucial del autoencoder, ya que, si este vector no posee suficiente calidad, no será posible generar buenos resultados en la reconstrucción.

Existen arquitecturas que utilizan este vector como entrada para una red neuronal adicional con el objetivo de generar nuevos vectores. Otras arquitecturas utilizan la predicción de la media y la desviación estándar para construir espacios latentes que sigan una distribución normal estándar.

C. Decoder

Conjunto de convoluciones encargadas de realizar upsampling, normalmente utilizando ConvTranspose2d en PyTorch con Python, con el objetivo de reconstruir la imagen a partir del vector latente con la mayor calidad posible.

Mientras más grande sea el vector latente, mejor podrá actuar el decoder; sin embargo, esto reduce el efecto de la reducción de dimensionalidad y trae consigo consecuencias como una mayor necesidad de almacenamiento.

VI. HÍPER PARÁMETROS A CONSIDERAR

- Tamaño de la codificación (vector latente): Mientras más pequeño sea el vector, mayor será la compresión obtenida; sin embargo, habrá menos información disponible para la reconstrucción. Esto requiere vectores latentes de muy alta calidad debido a la reducción en el margen de error permitido.
- Número de capas: Define la profundidad tanto del encoder como del decoder. A mayor cantidad de capas, más complejo será el modelo, aunque también requerirá un mayor tiempo de procesamiento y entrenamiento.
- Funciones de reconstrucción: Son funciones utilizadas para evaluar la calidad de la reconstrucción del modelo. Existe una gran variedad de ellas; sin embargo, las más comunes son las siguientes:
 - MSE (Mean Squared Error): Compara píxel por píxel las imágenes de entrada y salida. Presenta el inconveniente de generar imágenes con efecto de "blur", ya que utiliza el promedio del error de toda la imagen para propagar la pérdida durante el entrenamiento. Se utiliza normalmente en imágenes a color.
 - Binary Cross-Entropy: Calcula la probabilidad asociada a cada píxel. Debido a que se trabaja con valores normalizados de imágenes, suele producir imágenes más definidas y con menor efecto de "blur". Se utiliza comúnmente en imágenes en blanco y negro, así como en tareas de segmentación, contornos o generación de máscaras de objetos.

VII. TIPOS DE AUTOENCODERS

A. Undercomplete Autoencoder

Toma una imagen de entrada e intenta reconstruir la misma imagen en la salida, es el modelo "vainilla" que hemos estado viendo.

La dimensión del espacio latente es menor que la dimensión del espacio de entrada, obligando al modelo a aprender una representación comprimida de la información.

B. Sparse Autoencoder

Evita la reducción explícita de dimensionalidad en las capas ocultas, manteniendo dimensiones similares a lo largo de toda la arquitectura.

En lugar de ello, se penalizan ciertas activaciones para simular dinámicamente esta reducción. Para lograrlo, se aplican factores de regularización sobre la función de pérdida, provocando que algunas neuronas se activen y otras permanezcan inactivas dependiendo de la entrada.

De esta manera, diferentes regiones de la red son activadas selectivamente según las características de los datos de entrada.

C. Denoising Autoencoders

No constituyen una arquitectura completamente distinta; sin embargo, se consideran una categoría funcional enfocada específicamente en la limpieza y restauración de imágenes.

En este caso, no se compara directamente la entrada con la salida. En cambio, la salida del modelo se compara con la imagen original antes de haber aplicado los filtros o el ruido, permitiendo entrenar el autoencoder para recuperar la versión limpia de la imagen.

D. Variational Autoencoder

Los autoencoders convencionales no garantizan que el espacio latente sea continuo, lo que dificulta la interpolación. La interpolación consiste en estimar valores intermedios entre dos puntos conocidos; sin embargo, debido a la falta de continuidad en el espacio latente, las predicciones generadas entre muestras pueden resultar inconsistentes o poco representativas.

Los Variational Autoencoders, por otro lado, modelan el espacio latente mediante distribuciones de probabilidad. Esto permite obtener un espacio continuo para cada variable latente, facilitando modificar sus valores libremente para generar nuevas imágenes, principio fundamental detrás de gran parte de la IA generativa.

Este tipo de arquitecturas produce parámetros estadísticos, específicamente una media y una desviación estándar. Lo esperable es que imágenes similares generen medias y desviaciones estándar similares.

Con estos parámetros se realiza un proceso de reparametrización, el cual consiste en utilizar dichos valores para aproximar una distribución normal. A partir de esta distribución se obtiene una muestra z , correspondiente al vector latente.

La función de pérdida en este tipo de modelos es compuesta. Por un lado, se utiliza Binary Cross Entropy para asegurar una buena calidad de reconstrucción con respecto a la imagen de entrada. Por otro lado, se utiliza Kullback-Leibler Divergence

(KLD), la cual compara dos distribuciones de probabilidad para medir qué tan similares son, utilizando la media y desviación estándar obtenidas frente a una distribución gaussiana estándar.